

# ThreatFlow

Real-Time Security Anomaly Detection Pipeline

Kafka + Spark Structured Streaming + PostgreSQL + Grafana

*Working reference & status document — not the final report*

Module: Architecture et Technologies BigData — Master Sécurité IT & BigData

June 9, 2026

**Purpose of this document.** A living checklist and command reference for building the pipeline. Use it to track what is done, what remains, and to copy the exact commands for booting and verifying each stage. The graded written report is a separate deliverable (its structure is outlined in Section 7).

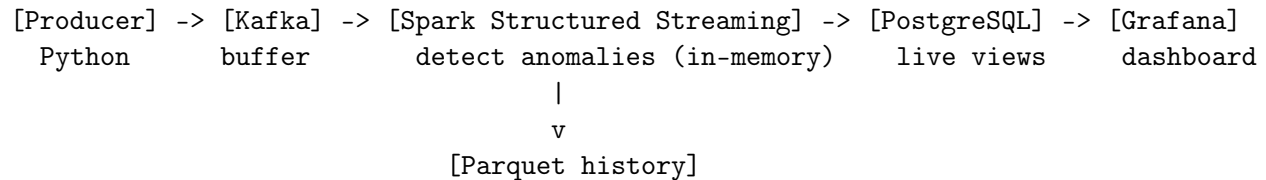
## Contents

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Project Overview</b>                                          | <b>2</b> |
| <b>2</b> | <b>Tech Stack</b>                                                | <b>2</b> |
| <b>3</b> | <b>Repository Structure</b>                                      | <b>2</b> |
| <b>4</b> | <b>Build &amp; Run</b>                                           | <b>3</b> |
| 4.1      | One-command launch . . . . .                                     | 3        |
| 4.2      | Dashboard-only (visualization teammate) . . . . .                | 3        |
| 4.3      | Stop / reset . . . . .                                           | 3        |
| 4.4      | Service endpoints . . . . .                                      | 3        |
| <b>5</b> | <b>Verification Commands</b>                                     | <b>3</b> |
| 5.1      | Containers healthy . . . . .                                     | 3        |
| 5.2      | Postgres schema created . . . . .                                | 3        |
| 5.3      | Kafka topic + live messages . . . . .                            | 4        |
| 5.4      | Producer is streaming . . . . .                                  | 4        |
| 5.5      | Spark job running . . . . .                                      | 4        |
| 5.6      | Data landing in Postgres (the end-to-end proof) . . . . .        | 4        |
| 5.7      | Parquet history written . . . . .                                | 4        |
| <b>6</b> | <b>Task Board</b>                                                | <b>4</b> |
| 6.1      | Infrastructure & Pipeline (lead) . . . . .                       | 4        |
| 6.2      | Dashboard (visualization teammate) . . . . .                     | 5        |
| 6.3      | Deliverables . . . . .                                           | 5        |
| <b>7</b> | <b>Report Structure (graded deliverable)</b>                     | <b>5</b> |
| <b>8</b> | <b>Known Issues &amp; Fixes (for the “Difficulties” section)</b> | <b>5</b> |
| <b>9</b> | <b>References</b>                                                | <b>5</b> |

## 1 Project Overview

**Goal.** Build a 100% open-source Big Data pipeline that ingests a real-time stream of security events, detects anomalies in near real-time with Apache Spark, and visualizes alerts on a live dashboard.

**Architecture: Kappa.** A single streaming path (no separate batch layer), chosen for low latency — the right fit for intrusion detection.



**Data source (chosen).** Synthetic generator — a Python producer fabricates authentication/network events and deliberately injects anomalies (failed-login bursts, blacklisted IPs). Chosen for control: guarantees detectable anomalies during the live demo.

**Detection rules.**

- `failed_login_burst`: more than 5 failed logins from one IP in a 1-minute window.
- `blacklisted_ip`: any event originating from a known-bad IP.

## 2 Tech Stack

| Component                | Role                                           |
|--------------------------|------------------------------------------------|
| Docker Compose           | Orchestrates all services on one host          |
| Confluent cp-kafka 7.6.1 | Message ingestion (pub/sub buffer, KRaft mode) |
| Apache Spark 3.5.1       | Real-time stream processing (PySpark)          |
| Python 3.11 (container)  | Synthetic event producer                       |
| PostgreSQL 16            | Storage of live views / alerts                 |
| Parquet                  | Raw replayable history                         |
| Grafana 11.1             | Real-time dashboard                            |

*Note on images:* Bitnami public tags were deprecated mid-project, so we switched to Confluent Kafka and the official Apache Spark image. The Spark image is customized to bake in the Kafka + Postgres connector JARs at build time (no runtime Maven download).

## 3 Repository Structure

```

threatflow/
|-- docker-compose.yml      # all services, one command
|-- README.md
|-- producer/
|   |-- producer.py         # synthetic generator -> Kafka
|   |-- Dockerfile          # pinned Python 3.11
|   |-- requirements.txt     # kafka-python-ng
|-- spark/
|   |-- stream_processor.py  # Kafka -> detect -> Postgres + Parquet
|   |-- Dockerfile          # Spark + baked-in JARs
|   |-- entrypoint.sh       # waits for Kafka, auto-submits
|   |-- init_db.sql         # Postgres schema (auto-loaded)
|   |-- .dockerignore
|-- dashboard/
  
```

```
| |-- seed_data.py           # fills Postgres with fake data (dashboard dev)
| |-- seed_requirements.txt
| '-- grafana/
|     |-- PANEL_QUERIES.md   # SQL for every Grafana panel
|     '-- provisioning/      # auto-configures Postgres datasource
|-- data/parquet/           # raw event archive (runtime)
'-- docs/                   # this document, diagrams, report, slides
```

## 4 Build & Run

### 4.1 One-command launch

```
docker compose up -d --build
```

Builds the producer and custom Spark images, then boots Kafka, producer, Postgres, Spark (auto-submits the job), and Grafana in dependency order.

### 4.2 Dashboard-only (visualization teammate)

No Kafka/Spark needed — build Grafana panels against seeded fake data.

```
docker compose up -d postgres grafana
pip install -r dashboard/seed_requirements.txt
python dashboard/seed_data.py # re-run anytime for fresh data
# open http://localhost:3000 (admin / admin)
```

### 4.3 Stop / reset

```
docker compose down # stop, keep data
docker compose down -v # stop and wipe Postgres + Grafana volumes
```

### 4.4 Service endpoints

| Service               | URL / Port                                                | Credentials   |
|-----------------------|-----------------------------------------------------------|---------------|
| Grafana               | <a href="http://localhost:3000">http://localhost:3000</a> | admin / admin |
| Postgres (host)       | localhost:5433                                            | secu / secret |
| Postgres (in-network) | postgres:5432                                             | secu / secret |
| Kafka (host)          | localhost:9092                                            | —             |
| Kafka (in-network)    | kafka:9094                                                | —             |

## 5 Verification Commands

### 5.1 Containers healthy

```
docker compose ps
# kafka + postgres should show (healthy); producer + spark Up; grafana Up
```

### 5.2 Postgres schema created

```
docker compose exec postgres psql -U secu -d securitydb -c "\dt"
# expect: alerts, event_counts
```

### 5.3 Kafka topic + live messages

```
docker compose exec kafka kafka-topics --bootstrap-server localhost:9092 --list
docker compose exec kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 --topic security-events \
  --max-messages 5
```

### 5.4 Producer is streaming

```
docker compose logs -f producer
# expect event stream + ">>> injecting brute-force burst" lines
```

### 5.5 Spark job running

```
docker compose logs -f spark
# expect: "Kafka is reachable. Submitting Spark job."
# "[spark] streaming started. Waiting for events..."
# batch lines: "wrote N alert(s)"
```

### 5.6 Data landing in Postgres (the end-to-end proof)

```
docker compose exec postgres psql -U secu -d securitydb \
  -c "SELECT rule, count(*) FROM alerts GROUP BY rule;"
docker compose exec postgres psql -U secu -d securitydb \
  -c "SELECT action, sum(count) FROM event_counts GROUP BY action;"
```

### 5.7 Parquet history written

```
ls -la data/parquet/
```

## 6 Task Board

### 6.1 Infrastructure & Pipeline (lead)

- ✓ **Done** — Docker Compose with Kafka, Postgres, Spark, Grafana
- ✓ **Done** — Switch off deprecated Bitnami images
- ✓ **Done** — Postgres schema auto-load (`init_db.sql`)
- ✓ **Done** — Containerize producer (Python 3.11)
- ✓ **Done** — Custom Spark image with baked-in JARs + auto-submit
- ◦ **In progress** — Confirm end-to-end: alerts & counts landing in Postgres
- □ **Pending** — Tune anomaly thresholds / inject more varied anomalies

## 6.2 Dashboard (visualization teammate)

- ✓ **Done** — Grafana auto-provisioned Postgres datasource
- ✓ **Done** — Seed script for offline dashboard development
- □ **Pending** — Build the 6 panels (see PANEL\_QUERIES.md)
- □ **Pending** — Configure Grafana alert rules
- □ **Pending** — Export dashboard JSON into the repo
- □ **Pending** — Switch from seed data to live pipeline output

## 6.3 Deliverables

- ◦ **In progress** — Source code in Git repo
- □ **Pending** — Written report (PDF + Word), 10–15 pages
- □ **Pending** — Presentation slides
- □ **Pending** — Backup demo video / screenshots
- □ **Pending** — Annotated architecture diagram

## 7 Report Structure (graded deliverable)

The written report ( $\approx$ 10–15 pages) must follow this plan:

1. **Cover page** — names of the 3 students, title, module, date.
2. **Introduction & context** — real-time security problem, project goal.
3. **Theory** — Batch vs real-time; Lambda vs Kappa architectures; roles of Kafka and Spark (with schema).
4. **Proposed architecture** — pipeline description, justification of choices, annotated diagram.
5. **Implementation** — data source, key commented code (producer, Spark, storage), screenshots.
6. **Results** — indicators obtained, anomalies detected, dashboard screenshots, latency observations.
7. **Difficulties & limits** — problems hit (Bitnami images, Python 3.13 / kafka-python, port clashes), workarounds, improvements.
8. **Conclusion** — summary and takeaways.
9. **Appendices & references** — code excerpts, links, dataset.

## 8 Known Issues & Fixes (for the “Difficulties” section)

- **Bitnami image tags unavailable** — bitnami/kafka:3.7 manifest not found (Bitnami migrated public images to a legacy repo). *Fix:* switched to confluentinc/cp-kafka and apache/spark.
- **Port 5432 already in use** — a local Postgres on the dev machine held the port. *Fix:* mapped the container to host port 5433.
- **kafka-python fails on Python 3.13** — No module named 'kafka.vendor.six.moves'. *Fix:* switched to the maintained fork kafka-python-ng, and later containerized the producer on pinned Python 3.11.
- **Maven JAR download fragility at runtime** — -packages downloaded connectors on every submit. *Fix:* baked the JARs into a custom Spark image at build time.

## 9 References

- Spark Structured Streaming guide — <https://spark.apache.org/docs/latest/structured-streaming->  
[html](https://spark.apache.org/docs/latest/structured-streaming-)

- Apache Kafka documentation — <https://kafka.apache.org/documentation/>
- Datasets (alternative sources) — <https://www.unb.ca/cic/datasets/>